The Hong Kong Polytechnic University

Database System Design Project

From Requirements to a Working MySQL Database

COMP2411 Database Systems

Project Guidance ▪ 2026

Project at a glance

Design, implement, test, and explain a relational database for an allocated real-world scenario. The project connects requirements analysis, ER modeling, relational design, normalization, SQL implementation, analytical queries, transactions, and justified index selection.

Instructor: HUANG Xiao

Academic Year 2026/27

Contents

1 Project Overview **2**
1.1 Key dates . . . 2

2 Learning Outcomes and Course Alignment **2**

3 Team Formation and Scope **3**
3.1 Permitted and recommended team sizes . . . 3
3.2 Join your team on Canvas . . . 3
3.3 Contribution and accountability . . . 3

4 Scenario Allocation for a Large Class **4**
4.1 Scenario ID structure . . . 4
4.2 Scenario difficulty and selection . . . 4
4.3 Registration procedure . . . 4
4.4 Student-proposed scenarios . . . 5

5 Stage 1: Proposal and Database Design **5**
5.1 Required contents . . . 5

6 Stage 2: Implementation and Demonstration **7**
6.1 Required files . . . 7
6.2 Minimum database evidence . . . 7
6.3 Index selection and justification . . . 8
6.4 Testing and analysis . . . 9
6.5 Demonstration requirement . . . 9

7 Assessment **10**
7.1 Stage weights and finality . . . 10
7.2 Stage 2 core rubric . . . 11
7.3 Common assessment structure and presentation . . . 12

8 Academic Integrity and Responsible Tool Use **12**

9 Submission Format and Reproducibility **13**
9.1 Archive names and directory structure . . . 13
9.2 Metadata and evidence formats . . . 14
9.3 Execution contract and README headings . . . 15
9.4 Transaction test execution . . . 15

10 Submission Checklist **16**

A Scenario Catalogues **17**
A.1 Domain seeds (D01–D25) . . . 17
A.2 Operational focus cards (W01–W08) . . . 18

B Scenario Fingerprint Template **19**

1 Project Overview

You will act as a database consulting team. Starting from a real-world operational scenario, you will identify data requirements, produce a conceptual design, transform it into a sound relational schema, and implement a working database. You must demonstrate that the database supports both day-to-day operations and useful management analysis.

The project contributes **20 marks** to the course assessment and has two assessed stages:

1. **Proposal and Database Design (5 marks)**: requirements, business rules, ER diagram, relational schema, and normalization.

2. **Implementation and Demonstration (15 marks)**: MySQL implementation, synthetic data, required SQL operations, testing, analysis, index justification, and documentation; all teams give an assessed in-person presentation and live demonstration.

Required preparation

All students must study `COMP2411_DBeaver_MySQL_Setup.pdf` and complete its installation and connection checks before implementation begins. **MySQL Community Server 8.4.11 is the standard DBMS and version for assessment. DBeaver Community is the recommended client.** Any alternative DBMS requires prior approval. DBeaver is the SQL client; MySQL Server stores and processes the database.

1.1 Key dates

Milestone	Deadline / date (2026)
Canvas group sign-up opens	**Thursday, 8 Oct, 16:00**
Canvas group sign-up and KCube Scenario ID registration deadline	**Monday, 12 Oct, 23:55**
Stage 1 submission	**Monday, 26 Oct, 23:55**
Stage 2 complete submission, including `presentation.pdf`	**Monday, 23 Nov, 23:55**
In-person presentation, live demonstration, and live-modification task	**Wed–Thu, 25–26 Nov**

All dates and times are in Hong Kong local time (UTC+8).

2 Learning Outcomes and Course Alignment

The project integrates requirements analysis, ER-to-relational mapping, keys and integrity constraints, functional dependencies and normalization, SQL DDL/DML and analytical queries, views, transactions, and index selection. You must justify design decisions, test correctness, and provide reproducible evidence using the concepts taught in lectures and labs.

3 Team Formation and Scope

3.1 Permitted and recommended team sizes

Teams must have **2–4 students**. One-student teams are not permitted; **four students** is the recommended team size.

Team size	Core project scope
2 students	The same required core deliverables as a four-person team.
3 students	The same required core deliverables as a four-person team.
4 students	Standard core scope. This is the **recommended** team size.

Requirement for all teams: A compulsory **in-person presentation and live demonstration**; all members must attend and participate in the group live-modification task and related questions (Section 6.5).

3.2 Join your team on Canvas

Canvas group sign-up opens on Thursday, 8 October 2026, at 16:00 (Hong Kong time). Complete **Canvas group sign-up and KCube Scenario ID registration** by **Monday, 12 October 2026, 23:55**. The course has 84 project groups. Each group's name includes its group number, presentation date, time, and room; **joining a group also selects that team's presentation slot**.

1. Agree on a team of 2–4 students and a presentation slot that every member can attend.
2. Open the COMP2411 Canvas Groups page (**People** → **Group**) and choose an available group. Read its complete name before joining.
3. **Every team member must join the same Canvas group**. One member joining does not register the other members. Check the displayed membership list together and remain within the 2–4 student limit.
4. Choose **one representative** to register the team's Scenario ID on KCube (Section 4.3). Joining a Canvas group does not register a project scenario.

Use the self-sign-up options available in Canvas. If section restrictions prevent your intended members from joining the same group, contact the teaching team before the registration deadline. Membership or presentation-slot changes after group sign-up require approval; do not independently switch groups to change your time.

Read the group name and use the correct Group ID

`Group 2, Nov 25, 8:36-8:41, TU103` means **Group 2** presents on **25 November 2026, starting at 08:36, in TU103**. The label covers the six scheduled minutes from 08:36 through 08:41; the next group starts at 08:42. All members of this group attend this same slot.
For submission filenames and metadata, write the group number as `G` followed by three digits: **Group 2** → `G002`. Likewise, Group 1 is `G001` and Group 84 is `G084`. This Group ID is distinct from the project's Scenario ID (for example, `D07-W03`) and from Canvas's internal numeric group identifier. Keep the same Group ID at both stages; do not put the date, time, or room in archive names.

3.3 Contribution and accountability

Every team submits a contribution declaration **signed by all members**. For each member, state the main responsibilities, identify relevant files or tasks, and select *Equal*, *Major*, *Minor*, or

No Contribution. Record any disagreement in the declaration; signatures acknowledge that all members have read it, not necessarily that every assessment is agreed. Individual marks for the core project may be adjusted when the declaration, working history, interview, or peer evidence shows substantially unequal contribution; the shared presentation score is not adjusted per member. The teaching team will explain the evidence and allow the affected member to respond before applying an adjustment. Keep working files and meaningful version history; the number of commits alone does not determine contribution.

4 Scenario Allocation for a Large Class

With approximately **300 students**, each team must register one unique **Scenario ID**. Teams selecting an available standard combination **do not need separate topic approval**. Students assess the difficulty themselves and define a manageable scope in Stage 1; scope feedback is provided through normal Stage 1 marking. Registration confirmation checks availability and records the choice; it is not an individual academic approval step.

4.1 Scenario ID structure

Each standard Scenario ID combines one of 25 **domain seeds** and one of eight **operational focuses**, excluding the four pairings listed in Appendix A:

$$\texttt{Dxx-Wxx}, \qquad 25 \times 8 - 4 = 196 \text{ selectable standard scenarios.}$$

For example, `D07-W03` means "Community Sports Centre + Capacity and Waiting-List Operations." The combination, not merely the broad domain, defines the project. Each exact Scenario ID may be registered by **one group only**. The 196 combinations differ in suitability and difficulty. If capacity is needed, the teaching team may publish additional workflow or policy variants with distinct IDs, such as `D07-W03-A`. Students must not invent a suffix to bypass registration.

4.2 Scenario difficulty and selection

Scenario combinations differ in complexity and workload. Assess your team's preparation and available time before choosing. Consider whether the domain and focus form a plausible workflow and whether it can satisfy the common project requirements. Successfully implemented and well-tested solutions involving more demanding business rules can earn higher marks for technical depth. Selecting a difficult scenario alone does not guarantee a higher mark.

Difficulty is judged from the completed work: interacting business rules, meaningful constraints, coherent transactions, and justified query logic. Table count, code length, and an unusual industry name are not measures of difficulty. The explicit technical-depth allocation is described in Section 7; there is no separate advance difficulty rating or approval for each standard topic.

4.3 Registration procedure

1. Form the team, have every member join the same Canvas group (Section 3.2), and review the catalogues in Appendix A, including the different workloads of possible combinations.

2. **One representative per group** registers an available Scenario ID on KCube, under **My Courses → SignUp**. In **Text response**, enter your **Canvas group number** (e.g., `Group 2`). If simultaneous requests conflict, the first complete registration takes priority; the other team selects another available scenario. The Scenario ID identifies the project topic; it does not determine the presentation slot.

3. Complete the **scenario fingerprint** in your Stage 1 proposal using the appendix template, including at least four management questions. Identify the business object, explain why the workflow fits, and state the scope, main challenges, and excluded processes.

4. Develop the proposal after registration; there is no separate approval gate for a standard combination. Stage 1 feedback may identify scope or design improvements for Stage 2.

5. The Scenario ID and core fingerprint are locked at Stage 1 submission. A topic change or substantial departure from the registered workflow requires written approval. Ordinary design corrections within the workflow are documented in the Stage 2 change log.

Originality rule

The same broad industry is allowed; the same Scenario ID is not. Changing only names, colours, or sample values does not create an independent design. Originality is assessed through independent design decisions, specific business rules, and their justification, not through how rare the industry topic is. A registered topic is not penalized merely because another project is in the same industry. The University Database used in lectures and a banquet/attendee database are not permitted. A trivial record list with little operational logic is also not acceptable.

4.4 Student-proposed scenarios

A group may request a custom scenario instead of using the catalogue. The request must identify its domain, workflow, users, non-trivial constraints, likely entities, and **at least four management questions**. Custom scenarios and substantial changes to registered scenarios require prior approval from the instructor or a TA. Approval considers scope and non-duplication; an approved custom project receives an ID such as `CUST-017`. Standard catalogue choices are not individually reviewed before Stage 1.

Use only fictional/synthetic records in the project database. Do not collect or submit real personal, medical, financial, employment, authentication, or location records as project data. Member names, student IDs, and signatures required for course administration are separate from the fictional database records.

5 Stage 1: Proposal and Database Design

Submission: one Stage 1 ZIP package per group through PolyU Canvas, containing `proposal.pdf`, `metadata.json`, and `ai_usage.md`. Use the exact naming convention in Section 9. The submission deadline is **Monday, 26 October 2026, 23:55**. A recommended proposal length is 8–12 pages excluding cover, references, and appendices; clarity matters more than length. Stage 1 is worth **5 marks**; subsequent revisions do not change its finalized mark.

5.1 Required contents

Deliverable	Required evidence
Identity and scope	Title, Group ID, full Canvas group name, Scenario ID, team size, and member names/student IDs matching Canvas; completed fingerprint with the business object, workflow fit, scope, exclusions, and difficulty rationale.
Requirements	At least **six precise business rules**, including cardinality, participation, uniqueness, and value constraints where relevant; **four operational use cases** that create, change, or retrieve data; and **four management questions**, including an aggregate question and a question requiring at least three tables. Each rule must affect the model, SQL, or testing.

Deliverable	Required evidence
ER model	A legible diagram showing entities, attributes, candidate/primary keys, named relationships, cardinality, and participation. Model weak entities and multivalued, composite, or derived attributes correctly when used; justify advanced constructs such as ternary relationships, specialization, aggregation, or associative entities.
Relational design	Consistent ER-to-relational mapping, including many-to-many relationships and weak entities. A data dictionary for every table: attributes, MySQL types, primary/candidate and foreign keys, nullability, defaults, and domain constraints.
Normalization	Important non-trivial functional dependencies, candidate keys, and the initial normal form; decomposition to at least 3NF (BCNF where appropriate), or justification that the target form already holds. Discuss lossless join and dependency preservation for non-obvious decompositions. Do not invent dependencies to force decomposition.
Traceability and plan	A compact matrix linking major requirements to ER constructs, relations/constraints, enforcement method or limitation, and planned tests; a division-of-work plan and main implementation risks.

Example of a management question

A management question is a business question that database analysis can answer to support a management decision. For an equipment-loan system:

- **Question:** Among loans returned in the past three months, which equipment categories had the most loans and the highest late-return rates?
- **Tables:** `Category`, `Equipment`, and `Loan`, linked through their primary and foreign keys.
- **Analysis:** Join the three tables, filter by return date, and group by equipment category. Count returned loans and late returns (return time later than due time); calculate the late-return rate as late returns divided by returned loans.
- **Decision supported:** Identify categories whose borrowing rules or reminder arrangements deserve review. For example, 30 late returns out of 120 returned camera loans gives a 25% late-return rate in synthetic data; this alone does not establish why borrowers returned equipment late.

This example combines aggregation with a three-table join. It illustrates the expected specificity; each team must formulate at least four management questions suited to its own scenario.

Scope boundary for all scenarios: Implement the focus card's minimum core requirement (Appendix A) within a manageable scope. Distinguish rules enforced by database constraints, checked through SQL operations, and left as documented limitations. Concurrent-user handling and automatic scheduling are not required; limitations do not waive the common or workflow requirements.

A typical design has about **5–8 meaningful, connected tables**. This is a planning guideline, not a hard limit; justify a smaller or larger design. Unnecessary tables do not earn marks.

6 Stage 2: Implementation and Demonstration

Submit one complete Stage 2 ZIP package per group, including `presentation.pdf`, through PolyU Canvas by **Monday, 23 November 2026, 23:55**, following Section 9. Stage 2 is worth **15 marks**. Implement the registered workflow using an improved final design, and explain changes from the submitted Stage 1 design in a change log. Improvements are assessed only as part of Stage 2; they do not reopen the Stage 1 mark.

6.1 Required files

1. **README.md:** a text-based user guide using the headings and execution conventions in Section 9.3; include software versions, setup order, account prerequisites without passwords, key operations, test instructions, and limitations.

2. **SQL scripts:**

   - `sql/01_schema.sql`: tables, keys, foreign keys, and other constraints;
   - `sql/02_data.sql`: synthetic seed data in a valid insertion order;
   - `sql/03_views.sql`: the required view definition(s);
   - `sql/04_indexes.sql`: additional index definitions, or comments identifying and justifying an existing index;
   - `queries/Q01.sql` through `queries/Q08.sql` (and higher IDs if needed): distinct business queries; and
   - `tests/`: successful operations, commit/rollback demonstrations, and separately executable negative tests.

3. **report.pdf:** final scope, business rules and management questions; final ER diagram, schema and data dictionary; normalization summary, change log, test cases/results, at least two analytical findings, and index-selection rationale.

4. **contribution.pdf:** responsibilities, contribution category, any disagreement, and signatures of all members.

5. **metadata.json, ai_usage.md, and evidence/:** machine-readable project information, the AI assistance declaration, query/test mapping files, and captured query results, using the formats in Section 9.2 and the AI declaration requirements in Section 8.

6. **presentation.pdf:** required for all teams; used in their in-person presentation and live demonstration.

7. **Application source code (optional):** include it only if your group builds an interface. A front end does not replace any required SQL evidence.

The four construction scripts in `sql/` must succeed in order in an empty project database. Query files and tests are executed separately after construction. Expected-failure tests are not part of installation. Use comments and the fixed filenames; do not submit a real password in any file.

6.2 Minimum database evidence

The final system must include:

- primary keys and foreign keys for all relevant relationships;

- meaningful `NOT NULL`, `UNIQUE`, `CHECK`, and referential actions where supported and appropriate;
- sufficient synthetic rows to exercise every relationship, constraint, query, and edge case;
- successful insert, update, and delete operations, plus deliberate invalid operations showing that important constraints reject bad data;
- at least **eight** labelled business queries in total, including:
  - two distinct queries with multi-table joins, with at least one using three or more tables;
  - two distinct aggregate queries using `GROUP BY`, with at least one using `HAVING`;
  - two distinct subquery-based queries; at least one must use a correlated subquery, `EXISTS`, or `NOT EXISTS`;
  - one query that handles missing values correctly; and
  - one additional non-trivial query chosen for the scenario;
- at least one useful `VIEW`;
- one multi-statement transaction with `COMMIT` and a demonstrated failure/rollback case; and
- one justified index, together with its target query and an explanation of the indexed columns, expected benefit, additional storage, and update-maintenance cost.

Counting queries: Label distinct business queries `Q01`–`Q08` (or higher) and map them to questions and techniques in `evidence/query_map.csv`. Technique categories may overlap, but eight distinct queries are still required; near-duplicates do not demonstrate breadth. Several queries may address one management question, so eight questions are not required. `CREATE VIEW` does not count; a business query using a view may count. The two analytical findings may reuse these results.

Data coverage: Include meaningful relationships and groups, relevant statuses/dates, matching and non-matching cases, legitimate missing values, groups retained and rejected by `HAVING`, and data that verify subqueries. State each query's expected result; most should be meaningfully non-empty, with intentional emptiness explained. Boundary and negative tests must reflect business rules; row count alone earns no marks.

6.3 Index selection and justification

Choose one representative query from your project and create an appropriate additional index using `CREATE INDEX`, or justify an existing index if it already serves that query. In `sql/04_indexes.sql`, define the new index or identify the existing index in SQL comments, and cite the target Query ID. Keep the executable target query in its `queries/` file. In the report, briefly explain:

- which query condition or join the indexed column(s) support;
- why the index may help locate the required rows, considering how many rows the query is expected to match; and
- the additional storage and maintenance work when rows are inserted or deleted, or indexed values are updated.

Assessment boundary: Explain the expected benefit using the indexing concepts taught in class. You do not need to prove a speedup, and should not claim a measured improvement without measurements.

6.4 Testing and analysis

Provide a test table containing test ID, requirement/rule, setup, SQL or action, expected result, actual result, and pass/fail status. Use the machine-readable format in Section 9.2 and discuss important results in the report. Include normal cases, boundary cases, null handling, duplicate-key attempts, invalid references, and transaction failure. Screenshots may support the report, but executable SQL and captured results are the primary evidence.

Separate success from expected failure. Run negative tests individually, outside construction, with one expected-failure statement per file under `tests/negative/`; the transaction rollback script is the documented exception. Identify each test, prerequisites, and expected MySQL error number or SQLSTATE; verify that rejection leaves the relevant data unchanged. Start each test from fresh seed data unless exact prerequisites are documented. Record concrete before/after evidence.

6.4.1 Transaction acceptance criteria

Using transactional tables (normally `InnoDB`), demonstrate a coherent business operation with at least **two related data-changing statements**:

- **Commit path:** explicitly start a transaction, perform the changes, and commit; before/after evidence must show that all intended changes persist.
- **Failure/rollback path:** explicitly start a transaction, make at least one successful change, then encounter a related failure and explicitly execute `ROLLBACK`; before/after evidence must prove that all earlier changes in that transaction were undone.

Submit both transaction scripts. Follow the same-connection, execution-block, and rollback instructions in Section 9.4; observing an error alone is not proof of rollback.

Present at least **two** decision-relevant analytical findings, each linking a management question to SQL, output, and a brief interpretation. Do not infer causation or business impact unsupported by the synthetic data.

6.5 Demonstration requirement

All teams must give an in-person presentation and live demonstration, including the live-modification task, on **25–26 November 2026 (Wednesday–Thursday)**. Your **Canvas group name is your presentation appointment**: it gives the exact date, time, and room for all members of that team. Check the current name and any teaching-team announcements before attending.

Groups	Date (2026)	Session starts	Room
1–28	Wed, 25 November	08:30	TU103
29–56	Wed, 25 November	15:30	TU101
57–84	Thu, 26 November	12:30	TU101

The table gives the first group's start time in each session. **Use the exact time in your own Canvas group name**, not just the session start. All times are Hong Kong local time in 24-hour format. Group numbers are presented in order within each session, with start times **six minutes apart**.

Six-minute assessment: four minutes plus two minutes

Each group has **4 minutes for its prepared presentation/live demonstration, followed by 2 minutes for one live-modification task and related questions**. There is no separate changeover time. The schedule is tight: be ready before your start time and finish before the next group's slot begins.

All members must attend and show student identification when requested. Be ready outside the assigned room **at least 10 minutes before your group's start time**. Bring the submitted slides and SQL scripts, and a working local MySQL database; test the connection and prepare the relevant queries in advance so setup does not consume the assessed slot. Contact the teaching team promptly about an attendance conflict; do not independently switch Canvas groups after group sign-up.

In the prepared portion, explain key design decisions, run representative SQL against the submitted database, and show correctness evidence, including a transaction or an important constraint test. Select a concise demonstration that fits the four-minute prepared portion. Use the submitted `presentation.pdf`; slides alone are not a live demonstration.

Two-minute live task: The instructor reveals one small modification or test only at the session. It is based on the group's submitted database, uses only taught material, and is designed for the two-minute slot. Tasks across groups will be of broadly comparable difficulty. Possible task types include:

- modify a query to also include customers with no bookings;
- adapt a report to count only completed orders and include only groups whose total amount exceeds a specified threshold;
- attempt to insert data that violate an enforced business rule, predict the outcome, and explain why the database rejects the operation; or
- modify one step of a transaction, demonstrate rollback after failure, and identify which data must be restored.

These are illustrative task types, not advance disclosure of the actual task. The instructor adapts the task to the group's schema and workflow; each group completes **one task, not all four**.

Permitted resources and participation: Students may consult their own SQL and course materials. During the in-person assessment, **AI tools, messaging/chat tools, and remote assistance are not permitted**. All members must participate in the group task and related discussion; the instructor may ask different members to explain or operate the database. All three presentation criteria remain **group-level assessments**, with one shared presentation score and no individual presentation or live-task marks (Section 7).

The teaching team may invite any group to a separate short verification interview when authorship, contribution, or reproducibility needs clarification. This interview is not an additional scored presentation component. Contact the teaching team promptly if an exceptional circumstance prevents attendance; alternative arrangements require approval.

7 Assessment

7.1 Stage weights and finality

The project contributes **20 marks** to the course: **Stage 1 = 5 marks** and **Stage 2 = 15 marks**. Once a stage mark is finalized, later improvements do not change it. Students may use Stage 1 feedback to improve Stage 2, but cannot resubmit Stage 1 for additional marks. Academic-integrity penalties are handled separately and can affect the whole project.

Stage 1 uses the same rubric for all team sizes:

Stage 1 criterion	Marks
Requirements, feasible scope, business rules, and independent design rationale	1
ER model, relational mapping, keys, and data dictionary	3
Functional dependencies and normalization justification	1
Stage 1 total	**5**

7.2 Stage 2 core rubric

The following common rubric produces a core score R out of 100. It assesses the *final submitted system*, including corrections made after Stage 1; it does not revise the Stage 1 mark.

Criterion	Evidence considered	Core weight
Requirements, originality, and scope	Registered Scenario ID/fingerprint; stakeholders; precise scope; specific rules and at least four management questions; independent design and rationale, not topic rarity.	12%
Conceptual and relational design	ER correctness; keys; cardinality/participation; ER-to-relational mapping; data dictionary; consistency and justification.	18%
Normalization and integrity	Functional dependencies; candidate keys; 3NF/BCNF reasoning; lossless/dependency discussion; implemented constraints.	12%
SQL implementation and functionality	Correct, reproducible schema/data scripts, DML, joins, aggregation, subqueries, view, and transaction (22%); successfully implemented and tested technical depth (6%).	28%
Testing, data, and analysis	Synthetic-data fitness; positive/negative/edge tests; analytical reports; correct interpretation.	12%
Index selection and justification	Appropriate index for a representative query; rationale for indexed columns; expected benefit; storage and update-maintenance trade-offs.	6%
Documentation and reproducibility	README; required archive structure and metadata; change log; readable outputs; reproducible construction and test instructions.	12%
	Total	**100%**

Technical depth (6 of the 28 percentage points for SQL): 0–2 for limited reasoning or incomplete evidence; 3–4 for coherent interacting rules supported by suitable SQL, constraints, and tests; 5–6 for demanding, justified rule interactions with convincing boundary/failure tests and correct transaction/query logic. These qualitative bands reward demonstrated reasoning, not feature count or topic labels. The remaining 94 points follow the common criteria. Untaught features, concurrency, and automatic scheduling are not required.

Marks reward correctness and justified decisions, not feature count alone. A smaller coherent design can outperform a large inconsistent one. A project that cannot be recreated from its submitted scripts may lose implementation and reproducibility marks according to the affected criteria.

7.3 Common assessment structure and presentation

Team size	Stage 1	Stage 2 core	Presentation
2 students	5 marks	12 marks	3 marks
3 students	5 marks	12 marks	3 marks
4 students (recommended)	5 marks	12 marks	3 marks

For every team, Stage $2 = 12R/100 + P$, where P is the group's shared presentation score out of 3. Thus presentation is **15% of the full 20-mark project**, or **20% of Stage 2**. The core rubric has the same relative proportions for every team size.

Presentation evidence (assessed for the whole group)	Marks
Accurate explanation and justification of the design and business rules	1
Successful SQL execution and correct interpretation during the prepared live demonstration	1
Group performance on the on-the-spot modification/test: correct changes or test, result interpretation, and explanation	1
Total shared presentation score	**3**

Presentation assesses the group's demonstrated database understanding, not speaking style. All three criteria are assessed at group level, and **every member receives the same presentation score** P. The instructor may ask different members to explain or operate the database, but this does not create individual presentation, live-task, or question-and-answer marks. Any approved alternative arrangement is assessed using equivalent group-level evidence. Contribution-based adjustments apply only to core-project marks, not to the shared presentation score.

8 Academic Integrity and Responsible Tool Use

The submitted design, SQL, explanation, and demonstration must represent the group's own work. Cite external datasets, diagrams, code, and documentation. Do not reuse another group's current or past project, purchase a solution, or share scripts between groups.

AI assistance during project preparation is allowed; outsourcing authorship is not. You may ask AI tools for conceptual explanations, debugging help, or suggestions about a specific issue. This permission does not apply during the in-person assessment, where the resource restrictions in Section 6.5 apply. The main design, core implementation, and report must remain the group's own work. Verify all advice against your database and be able to explain and modify the submitted SQL. Cosmetic rewriting, renaming identifiers, or changing a few values does not turn AI-generated work into independent authorship.

At both stages, submit `ai_usage.md`. Identify the tool, purpose of assistance, affected files/sections, what was accepted or rejected, and how the group checked it. If no AI was used, state "No AI assistance was used." Keep drafts and working history as evidence of the development process. Citing AI assistance does not make substantially outsourced work acceptable.

Whole-project zero for predominantly AI-generated work

If the assessment review determines that the majority of the project's core code *or* report was generated by AI rather than authored by the group, the project receives **0 out of 20**, not merely zero for one stage. "Majority" concerns the substantive implementation or explanatory content, not just a raw line or word count.

A TA who suspects this breach will review concrete evidence, such as drafts, development history, the assistance declaration, and the group's explanation or live modification of its work. Students will have an opportunity to respond; the instructor confirms the decision before the penalty is applied. An AI-detector score, writing style, or one weak interview answer alone is not sufficient evidence. Other academic-integrity concerns follow the applicable course and University procedures.

9 Submission Format and Reproducibility

This section defines the required packaging and execution conventions for both stages. Examples illustrate submission formats, not a model project solution; replace all example identifiers and values with your own information.

One designated member submits one ZIP package on behalf of the whole group to the correct Stage 1 or Stage 2 assignment in Canvas by the relevant deadline. Canvas group sign-up and KCube scenario registration do not replace project submission. Before uploading, verify that every member appears in the same Canvas group and that the archive and metadata use its Group ID. After uploading, check the submission confirmation and uploaded filename, and make the submitted package available to all members.

9.1 Archive names and directory structure

Use the Group ID derived from your Canvas group number (Section 3.2) consistently at both stages.

For example, Group 2 submits `COMP2411_2026_G002_Stage1.zip` and `COMP2411_2026_G002_Stage2.zip`. Do not include the full Canvas group name, presentation details, member names, or Scenario ID in archive names. Each ZIP must contain exactly one top-level directory with the same name as the ZIP without `.zip`. Filenames and directory names below are case-sensitive. Use UTF-8 for text files and do not add extra wrapper directories, passwords, database binaries, caches, or editor dependencies.

```
COMP2411_2026_G002_Stage1/
  metadata.json
  proposal.pdf
  ai_usage.md

COMP2411_2026_G002_Stage2/
  metadata.json
  README.md
  report.pdf
  contribution.pdf
  ai_usage.md
  sql/
    01_schema.sql
    02_data.sql
    03_views.sql
    04_indexes.sql
  queries/
    Q01.sql ... Q08.sql
  tests/
    positive.sql
    transaction_commit.sql
    transaction_rollback.sql
    negative/
      N01.sql ...
  evidence/
    query_map.csv
    test_cases.csv
    query_results/
      Q01.csv ... Q08.csv
  presentation.pdf (required for all groups)
```

```
  app/ (optional interface source)
```

Ellipses indicate separate files, not literal filenames. Additional business queries continue with `Q09`, `Q10`, etc., each with a matching result CSV and mapping row. Number negative tests `N01`, `N02`, etc. The declaration in `contribution.pdf` must include all member signatures; scanned handwritten or electronic signatures are acceptable. Do not include signatures or student details in the synthetic database.

9.2 Metadata and evidence formats

`metadata.json` must be valid JSON, with the following required keys. Replace every example value with your own information; `stage` is 1 or 2 and `team_size` must be 2, 3, or 4 and match the number of members. The member list must match your Canvas group. Copy `canvas_group_name` exactly as displayed in Canvas, including its date, time, and room; derive `group_id` from its group number. Store student IDs as strings. Use `null` for `approval_reference` when no exception needs approval; otherwise identify the relevant written approval, including any approved membership or presentation-slot change, custom topic, topic change, or alternative DBMS.

```
{
  "group_id": "G002",
  "canvas_group_name": "Group 2, Nov 25, 8:36-8:41, TU103",
  "scenario_id": "D07-W03",
  "stage": 2,
  "team_size": 4,
  "members": [
    {"name": "Student A", "student_id": "REPLACE_A"},
    {"name": "Student B", "student_id": "REPLACE_B"},
    {"name": "Student C", "student_id": "REPLACE_C"},
    {"name": "Student D", "student_id": "REPLACE_D"}
  ],
  "dbms": "MySQL Community Server",
  "dbms_version": "8.4.11",
  "database_name": "comp2411_g002",
  "approval_reference": null
}
```

For Stage 1, the DBMS/version/database fields describe the planned environment; for Stage 2, record the actual verified environment. An approved alternative must identify its actual DBMS and version.

Show the Group ID, full Canvas group name, Scenario ID, member names, and student IDs on the proposal/report cover and the presentation title slide. Check the current Canvas group name before each submission. If the teaching team later updates a presentation slot, follow the updated Canvas arrangement; an earlier submitted metadata file is a record of the arrangement at submission time.

CSV files must have the exact header rows below. Quote fields that contain commas, quotes, or newlines using normal CSV escaping. Paths are relative to the archive's top-level directory and use forward slashes.

`evidence/query_map.csv`: one row per business query.

```
query_id,script,business_question,techniques,result_file
```

For `Q01`, use script `queries/Q01.sql` and result file `evidence/query_results/Q01.csv`. The business-question field identifies one of the proposal's questions, or a justified refinement in the change log. List applicable technique tags separated by semicolons: `join`, `join3plus`, `group_by`, `having`, `subquery`, `correlated`, `exists`, `null_handling`, `view`, or `other_nontrivial`. Include all applicable tags, not only the most advanced one. Explain the non-trivial logic when using the last tag.

evidence/test_cases.csv: one row per labelled test case; several cases may use labelled blocks in tests/positive.sql. Give each case a unique ID, such as POS01, TXC01, TXR01, or N01.

```
test_id,requirement,script,initial_state,expected_result,expected_error,actual_result,status
```

Use PASS or FAIL for status. Use NONE for expected_error in successful tests; otherwise give the MySQL error number or SQLSTATE. Specify concrete values/rows for expected and actual results, including unchanged state after rejection or rollback. Reference the corresponding Test ID in the SQL comments and any supporting report evidence.

Captured query results: each Qxx.sql contains one business SELECT statement (a statement may contain subqueries) with a Query ID comment; it must not modify data. Run it on the freshly constructed seed state, before tests change the data, and export its output to the matching Qxx.csv, including column headers. Use explicit column aliases where needed and stable seed values; avoid time-dependent or random outputs. If row order matters, specify ORDER BY; otherwise comparison is by row contents, retaining duplicates, not display order. Represent SQL NULL as the reserved text \N, keep empty strings empty, and do not use the literal string \N as synthetic data. For an intentionally empty result, submit the header row and explain the expected emptiness.

9.3 Execution contract and README headings

The evaluator creates an empty dedicated database and selects it as the active database before running the four construction scripts in numerical order. Record the intended database name in metadata. Use unqualified project table names. Do not submit CREATE/DROP DATABASE, account creation, server administration, global settings, machine-specific file paths, or real credentials in executable scripts. Document account prerequisites in the README instead. Do not disable foreign-key checks to hide invalid construction order. All project objects and data must be created by the submitted scripts without manual edits or an external import.

Use these exact top-level headings in README.md:

```
# Environment
# Build
# Queries
# Tests
# Limitations
# File inventory
```

Under **Environment**, state tested versions, privileges, and any approval reference. Under **Build**, list the four construction scripts and explain how to start from an empty dedicated database. Under **Queries**, describe the seed baseline and result-export convention. Under **Tests**, give the starting state, execution blocks, expected errors, and explicit rollback procedure; explain how to restore the seed baseline between tests. Under **Limitations**, distinguish implemented rules from SQL checks and documented exclusions. Under **File inventory**, list submitted files and their purposes. A missing test or a known failure must be disclosed, not hidden by labelling it as an expected error.

9.4 Transaction test execution

Section 6.4.1 defines the assessed commit and rollback outcomes. Use tests/transaction_commit.sql and tests/transaction_rollback.sql for those two paths. Start each test from a freshly constructed seed database unless its exact prerequisites are documented. Record concrete before/after values or row sets in evidence/test_cases.csv.

Mark the expected-error statement and the following rollback/verification block clearly in comments. Run these blocks interactively in the *same connection*; if the client stops on an error, continue with the rollback/verification block. Document this procedure and the connection's autocommit setting in README.md. Do not put DDL inside the demonstrated transaction, because MySQL DDL can

cause implicit commits. The rollback script is the explicit exception to the single-error-file rule for negative tests: its setup, failure, rollback, and verification form one transaction test. Concurrent sessions and automatic scheduling are not required.

Run `tests/negative/Nxx.sql` files separately, with one expected-failure statement per file and documented prerequisites, Test ID, and expected MySQL error number or SQLSTATE. Verify that each rejected operation leaves the relevant data unchanged. Expected failures must never be mixed into the construction scripts.

State any material SQL-dialect choices in the README. Use a course database account with the necessary privileges; no real credentials belong in the archive.

10 Submission Checklist

Before submission, check that:

- every member has joined the same Canvas group; its number matches the Group ID in the archive, metadata, and documents;
- the full Canvas group name, member details, and required approvals are correct, and the KCube Scenario ID registration includes your group number in **Text response**;
- all members have checked the presentation date, time, and room in the current Canvas group name and can attend;
- the stage deliverables and submission requirements in Section 9 are complete, including the AI declaration and all Stage 2 contribution signatures;
- diagrams, schema, SQL, and report agree; changes since Stage 1 are explained;
- Stage 2 rebuilds cleanly, with labelled queries, captured results, and independently reproducible tests;
- database records are synthetic; no real credentials or sensitive records are included; and
- the archive opens, PDFs are readable, and every member can explain the complete design.

A Scenario Catalogues

Excluded combinations: `D21-W01`, `D21-W03`, `D25-W03`, and `D25-W04` cannot be registered. The remaining **196 combinations** form the standard catalogue; domain and focus IDs are unchanged.

A.1 Domain seeds (D01–D25)

ID	Domain seed	ID	Domain seed
D01	Community health appointment centre (no real patient data)	D14	Repair and maintenance service centre
D02	Veterinary clinic operations	D15	Shared equipment and laboratory booking
D03	Elderly community support services	D16	Co-working space operations
D04	Child enrichment programme centre	D17	Local farm subscription and distribution
D05	Language-learning centre	D18	Food-rescue collection and allocation
D06	Museum education and guided tours	D19	Recycling collection programme
D07	Community sports centre	D20	Event venue operations (not banquet attendees)
D08	Outdoor activity provider	D21	Independent publishing workflow
D09	Performing-arts production company	D22	Media equipment rental and production
D10	Music rehearsal studio	D23	Residential property maintenance
D11	Pet boarding and grooming service	D24	Volunteer and community-service programme
D12	Bicycle-sharing operator	D25	Small manufacturer and quality inspection
D13	Vehicle service workshop		

A.2 Operational focus cards (W01–W08)

Each card states the workflow's **minimum core requirement**; meet it alongside the common project requirements within a manageable scope. Demonstrate the rules using database constraints or explicit SQL checks and operations; manual execution is acceptable. Concurrent-user handling, automatic scheduling, background services, and general workflow engines are not required.

ID	Focus	Minimum core requirement
W01	Reservation lifecycle	Manage requests through confirmation and cancellation or rescheduling, checking availability of a defined resource or service.
W02	Resource assignment	Assign resources to jobs or activities using explicit suitability and overlap checks, demonstrating acceptance of valid assignments and prevention of conflicting ones.
W03	Capacity and waiting list	Apply a defined capacity limit and waiting-list order, and demonstrate withdrawal followed by promotion when a place becomes available.
W04	Membership or subscription	Manage enrolment and renewal with explicit eligibility or validity rules, and record the use of defined benefits, entitlements, or quotas.
W05	Inventory and replenishment	Record stock receipts and issues, calculate current balances, and identify replenishment needs using a stated reorder rule.
W06	Case or work-order lifecycle	Track a case or work order from intake to completion, recording responsibility, permitted status transitions, and actions taken.
W07	Bundles and dependencies	Track a multi-part bundle or programme with a defined completion rule and at least one prerequisite or permitted-substitution rule.
W08	Quality, incident, and compliance	Record inspections or incidents against stated project-defined criteria, assign corrective actions, and track their resolution.

B Scenario Fingerprint Template

Complete this template within the existing Stage 1 proposal; no separate submission is required.

Group ID	`G___`
Full Canvas group name	Copy the complete name, including presentation date, time, and room.
Scenario ID	`D__-W__` or approved `CUST-___`
Organization name and type	
Principal users/roles	
Business object and core event	Identify what the workflow manages and the main business action.
Distinctive policy or constraint	
Management questions (at least four)	List Q1–Q4 or more; include an aggregate question and a question requiring at least three tables.
Excluded processes	State which processes the project will not implement.
Workflow fit and scope rationale	Explain why the workflow suits the business object, the main challenges, and why the chosen scope is manageable.